

Introduction to ASP.NET Core Web API

Back to: [ASP.NET Core Web API Tutorials](#)

Introduction to ASP.NET Core Web API

In this article, I will provide a brief introduction to the ASP.NET Core Web API Framework. ASP.NET Core Web API is a modern framework by Microsoft for building lightweight, fast, and cross-platform web services. It enables various applications, such as websites, mobile apps, and desktops, to communicate with a single backend using HTTP.

Why do we need Web APIs?

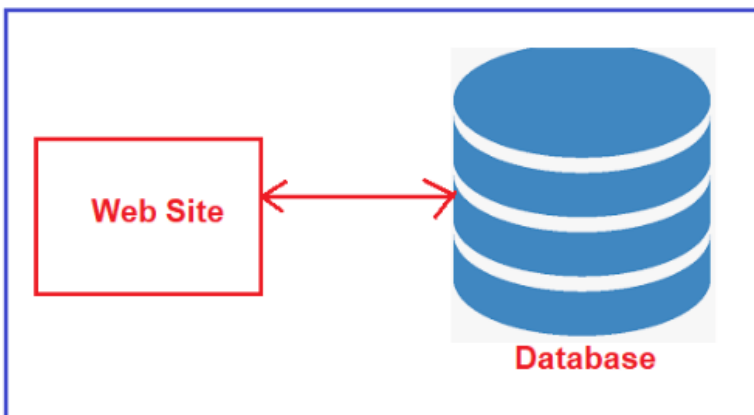
In today's multi-device world, Web APIs are the **backbone of modern applications**, powering websites, mobile apps, and even smart devices. Let us understand the need for Web APIs with some realistic examples.

Step 1 – Initial Setup (Only Website + Database)

Suppose you have an idea to develop and launch a product. For this, you need to develop a website and launch this product. Then what will you do? The simplest setup would be:

- A **Website** built using **ASP.NET MVC, ASP.NET Core, PHP, JSP**, or any other **server-side technology** that is available in the market.
- Of course, a **Database** such as **SQL Server, MySQL, or Oracle** to store all your product's business data.

With this setup, your website becomes a dynamic, fully functional system that can create, read, update, and delete data from the database.



Step 2 – Business Grows (Website + Android + iOS Apps)

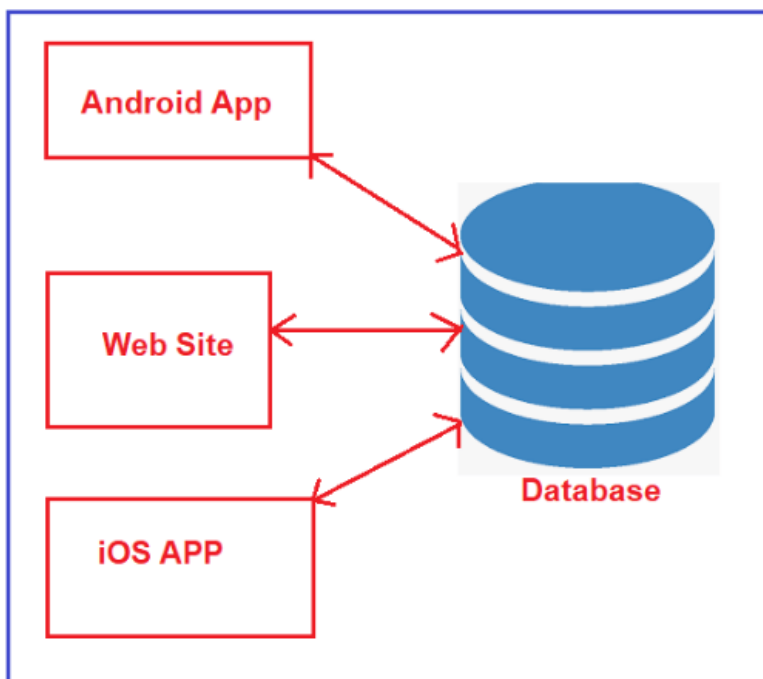
Over time, your business grows. Now, besides the website, you also want:

- An **Android Mobile App**
- An **iOS Mobile App**

This means:

- You now have **three different applications** (Website, Android, iOS).
- But you still have **only one database** that stores all your data.
- So, we have three different applications and one database.

Now, all these three applications have to interact with the database, as shown in the below image.



Step 3 – Problems With Direct Database Access

If all these three applications directly interact with the database, we have some problems. Let us understand the problems first, and then we will see how to overcome the above problems:

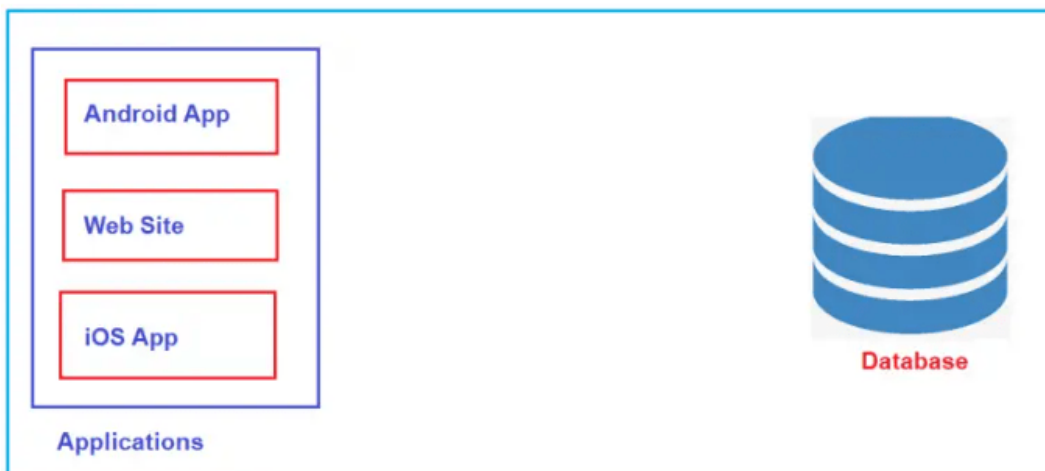
- **Duplicate Logic Across Applications:** Each application must implement the same business logic independently. This causes **Code Duplication**.
- **Time-Consuming and Error-Prone:** Writing the same logic in three places increases the chances of mistakes. If a piece of logic changes, you need to update it everywhere. This will add more errors to your application
- **Front-End Framework Limitations:** Some frameworks (like **Angular or React**) cannot directly talk to a database, because they are client-side technologies. They need a middle layer.

- **Hard to Maintain:** If you need to update or fix business logic, you must do so in multiple applications, which is a complex and inefficient process.

In short, **direct database communication is messy, unsafe, and hard to scale**. There are also other problems that we face in this structure. Let us see how to overcome the above problems, or, in other words, why we need Web APIs.

The Need for Web APIs

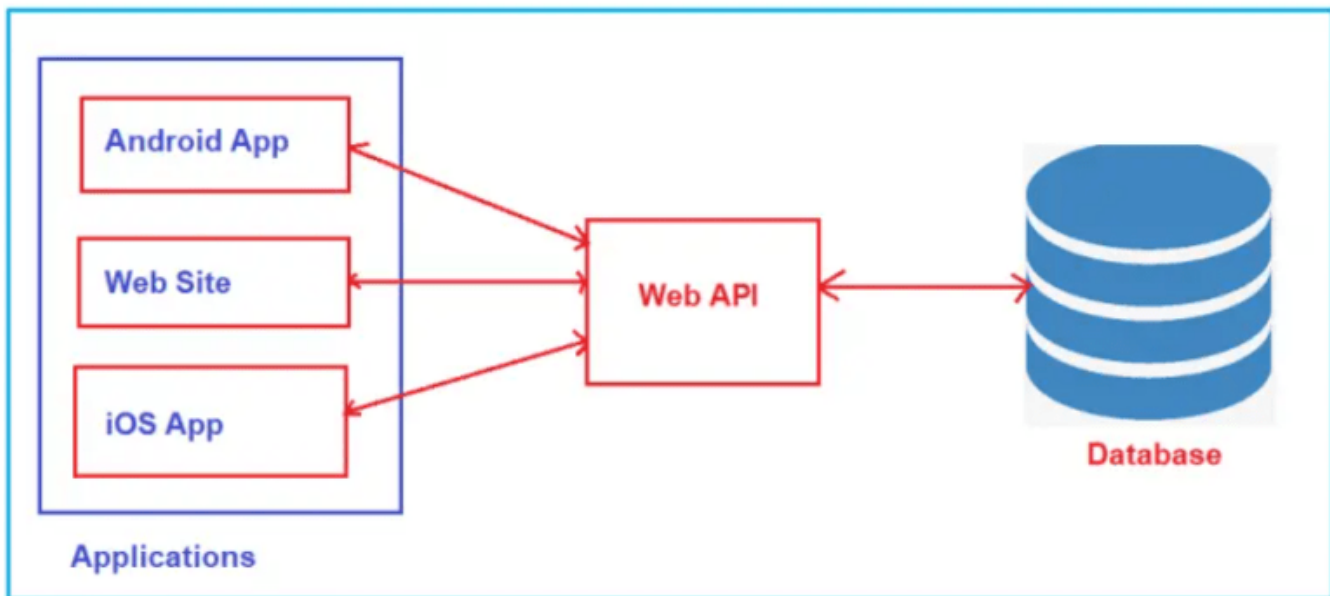
As you can see in the image below, we have three applications on the left-hand side, and on the right-hand side, we have the database.



We aim to establish communication between all three applications and the database to manage the data. So, what will we do? We will add a **Web API layer** between the **frontend applications (website, Android, iOS)** and the **backend database**.

- Instead of each application directly accessing the database, they all communicate with the **Web API**.
- The **Web API** contains all the **business logic** and manages all communication with the database.
- This makes the **Web API act as a mediator** between the frontend and the backend.

As a result, the database becomes secure, logic is centralized, and applications become easier to maintain and extend.



Note: The Website, Android, and iOS applications do not have direct access to the database. They only need to communicate with the Web API Project, and it is the Web API project's responsibility to interact with the database. The entire business logic will be written in the Web API project only. So, Web API acts as a mediator between the Front-End and Back-End.

Advantages of Web APIs

Platform Independence:

- Any application (Website, Android, iOS, desktop app, or IoT device) can consume the API as long as it understands HTTP protocols (such as HTTP Methods, HTTP Headers, and HTTP Status codes), regardless of the underlying programming language.
- **Example:** A .NET backend API can serve data to an iOS app built in Swift, a website built in Angular, or a React-based website, etc.

Low Bandwidth Usage:

- APIs typically exchange data using JSON or XML, which is much lighter than full HTML Pages.
- **Example:** An API can send just {"ProductId": 1, "Price": 100} instead of a full webpage with CSS/JS.
- This reduces the size of data transferred, making communication faster and efficient.

Reusability of Code:

- APIs promote code reusability.
- The business logic is written once in the Web API. Multiple clients (Website, Android, iOS, future apps) can reuse the same API endpoints.
- **Example:** Discount calculation logic is implemented only in the API. The website, Android, and iOS apps all use it via the API—ensuring consistency.

Improved Security:

- APIs can enforce **Authentication and Authorization** so that only valid users or applications can access sensitive resources.
- **Example:** Only logged-in users can place an order, while admins can access reporting endpoints.
- Direct database access from clients is eliminated, which adds layer of security.

Easy Extension of Application Functionality

- **Example:** You first build a **website** using the API.
- Later, you build an **Android app** → no need to rewrite logic, just consume the same API.
- In future, you want to build an **iOS app** → again, reuse the same API.
- Future integrations (like third-party partners, IoT devices, or external dashboards) also become possible.

What is Web API?

A **Web API (Web Application Programming Interface)** is a **bridge** that enables two different software applications to communicate with each other over the internet. Instead of directly connecting databases or systems, APIs define **Rules, Protocols, and Data Formats** that applications can use to send requests and receive responses. **In simple terms**, A Web API is like a waiter in a restaurant; it takes your order (request), passes it to the kitchen (server), and brings back the food (response).

Web APIs typically operate on top of **HTTP/HTTPS**, making them accessible from anywhere using standard web technologies. They provide **Endpoints (URIs)**, each mapped to a specific operation. These endpoints can be consumed by **web apps, mobile apps, desktop software, IoT devices, and third-party services**.

What is Web API

A Web API (Application Programming Interface) is a set of rules that enables different software applications to communicate with each other over the internet.

In simple terms: A Web API is like a waiter in a restaurant: it takes your order (request), passes it to the kitchen (server), and brings back the food (response).

- Communicates over HTTP/HTTPS
- JSON, XML, etc.
- Provides endpoints for applications



Example – Weather API:

- GET <https://Example.com/weather/today?location=Mumbai> → Returns today's weather for Mumbai.
- GET <https://Example.com/weather/forecast?location=Mumbai&days=5> → Returns a 5-day weather forecast for Mumbai.

Here, clients don't need to know the **internal database or code** of the weather service. They just send requests to well-defined endpoints and receive structured data.

Key Points:

- Communication happens over **HTTP/HTTPS**.
- Data formats include **JSON** (most popular), **XML**, **HTML**, or even plain text.
- Provides **Endpoints (URIs)** that represent resources or actions.
- Clients can be **web apps, mobile apps, desktop software, or IoT devices**.

What is ASP.NET Core Web API?

ASP.NET Core Web API is Microsoft's modern framework for building **HTTP-based services (RESTful APIs)** on top of the .NET Core platform. It enables developers to create lightweight, scalable, and secure APIs that multiple types of clients can consume.

According to Microsoft, it is the **Ideal Framework** for building RESTful services in the .NET ecosystem. Since it is built on **ASP.NET Core**, it inherits all the powerful features of the platform such as **Cross-Platform Support**, **Dependency Injection**, **Middleware Pipeline**, **Asynchronous Programming**, and **High Performance**.

What is ASP.NET Core Web API?

ASP.NET Core Web API is Microsoft's modern framework for building HTTP-based services (RESTful APIs) on top of the .NET Core platform.

These APIs can be consumed by:



Browsers



Mobile applications



Desktop applications



Console applications



IoT devices

These APIs can be consumed by:

- **Browsers:** AJAX requests or SPAs (Single Page Applications like Angular, React, Vue).
- **Mobile applications:** Android, iOS apps communicate via API.
- **Desktop applications:** Windows or cross-platform desktop apps can consume APIs.
- **Console applications:** For batch jobs or background tasks.
- **IoT devices:** Smart devices, wearables, or embedded systems can communicate with APIs.

In simple terms, ASP.NET Core Web API enables you to securely expose your business logic and data to any client that understands HTTP.

Why We Need ASP.NET Core Web API?

Today, businesses can't rely solely on a website because users access data from **multiple devices**, including **smartphones, tablets, smart TVs, IoT devices, and even voice assistants**. To support this **multi-device ecosystem**, you need a central way of exposing your business data and services.

That's where ASP.NET Core Web API comes in: it acts as a **single interface** between your business data and all the clients. **Reasons We Need It:**

1. **Multi-Device Access:** One API can serve browsers, mobile apps, and IoT devices simultaneously.
2. **Separation of Concerns:** Business logic lives in the API, while clients only consume the data.

3. **Scalability:** APIs can handle thousands/millions of requests if built on a high-performance framework like ASP.NET Core.

Advantages of ASP.NET Core Web API:

- **Performance:** ASP.NET Core is lightweight, modular, and optimized for high performance. Benchmarks consistently show it as one of the fastest frameworks, making it ideal for high-traffic APIs (e.g., e-commerce, social media apps).
- **Cross-Platform:** Unlike the old ASP.NET, which only worked on Windows, ASP.NET Core works on **Windows, Linux, and macOS**. This gives flexibility for deploying APIs on different environments (on-premises servers, cloud, or containers like Docker).
- **Modularity:** ASP.NET Core is built with a modular architecture. You only add the features you need (via NuGet packages). This keeps the API project lightweight and easier to maintain.
- **Modern Features:** It comes with features required in modern API development:
 - **Dependency Injection** → Manage application dependencies cleanly.
 - **Middleware Pipeline** → Customize request/response processing easily.
 - **Asynchronous Programming (async/await)** → Handle high-concurrency workloads efficiently.
- **Security:** Out-of-the-box support for:
 - **Authentication** (JWT, OAuth, Identity)
 - **Authorization** (Role-based, Policy-based access control)
 - This ensures your APIs are protected from unauthorized access.

In simple terms, ASP.NET Core Web API is needed because it allows businesses to **expose their data and services securely** to all kinds of devices in a **fast, modern, and scalable** way.

What is Rest?

REST (Representational State Transfer) is not a technology or a framework, but an **architectural style** for designing **Distributed Systems**. Its primary purpose is to define how **clients** (like browsers, mobile apps, or other applications) and **servers** (back-end services) should communicate over a network.

The beauty of REST lies in its **Platform Independence**:

- The client could be written in Java, .NET, Angular, React, or any other technology.
- The server could be built using Node.js, ASP.NET Core, PHP, Python, or Java.
- As long as they both follow REST principles and communicate via **HTTP**, they can exchange data seamlessly.

REST treats **Everything as a resource**: books, users, orders, products, etc. Each resource is uniquely identified by a **URI (Uniform Resource Identifier)**, and standard **HTTP methods** (GET, POST, PUT, DELETE, PATCH, etc.) are

used to perform operations (CRUD – Create, Read, Update, Delete) on these resources.

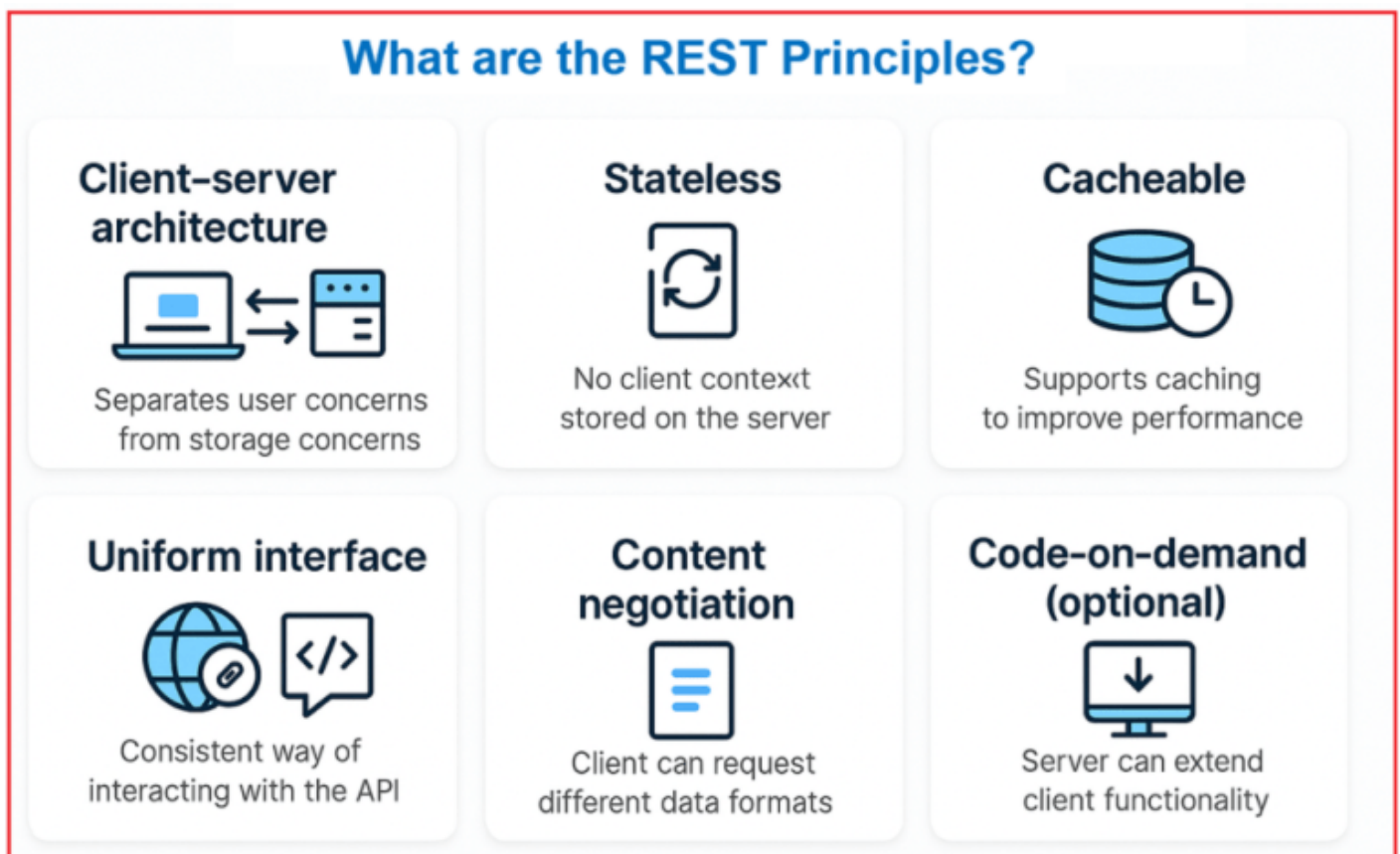
Example – Book Management System API:

- GET /books → Retrieve all books.
- GET /books/{id} → Retrieve details of a specific book.
- POST /books → Add a new book.
- PUT /books/{id} → Update details of a book.
- DELETE /books/{id} → Delete a book.

This clear and uniform way of accessing resources is what makes REST widely adopted in modern web APIs.

What are the REST Principles?

REST defines **six key principles (constraints)** that make a system truly RESTful. Let us proceed and understand the REST constraints or principles.



Client-Server Architecture:

REST enforces a clear separation of responsibilities:

- **Client** → Handles user interface, presentation, and experience.

- **Server** → Handles business logic, processing, and database/storage.

They communicate using a **standard interface (HTTP)**.

Why it matters:

- Client and server can be developed, deployed, and scaled **independently**.
- UI changes (say, redesigning a mobile app) don't affect the back-end logic.
- Server upgrades (such as database migrations) don't break clients.

Example: In Instagram, your mobile app (client) fetches profile data via the GET /users/{id} endpoint. The server retrieves the data and returns it, regardless of whether the request originated from iOS, Android, or a web browser.

Stateless:

The **stateless constraint** specifies that client-server communication must be **stateless** between requests. Each request from the client-server must be **independent**. The server does **not store any session data** between requests. Every request from the client should carry **all necessary information** (authentication, parameters, etc.) for the server to process it.

Why it matters:

- Any server in a cluster can handle any request (better scalability).
- Simplifies server design (no need to track client state).
- Improves reliability (no "session lost" issues).
- **Each request from the clients can be treated independently by the server.**

Example: When a client calls **GET /orders/789**, it sends an **Authorization token** in headers. The server validates the token, fetches the order, and responds. The server doesn't need to remember past interactions.

Cacheable:

In real-time applications, some data provided by the server is not changed frequently, such as the list of **Countries, States, Cities, and products**. REST allows clients (and intermediate proxies) to **cache responses**, **improving performance and reducing** server load. The server informs the client whether a response is cacheable and for how long, using headers such as Cache-Control or Expires.

Why it matters:

- Faster responses for frequently accessed data.
- Reduces redundant server calls.
- Optimizes bandwidth usage.

- Frequently requested resources can be served from a cache instead of re-fetching from the server.
- Reduces the number of round-trip requests to the server.

Example:

- A request GET /countries may rarely change. The server responds with:
- Cache-Control: max-age=86400
- This means the client can reuse the response for 24 hours without needing to request it again from the server.

Uniform Interface:

The most critical principle: all resources are accessed in a **consistent and standardized way**. This makes APIs predictable and easier to use.

It has four sub-constraints:

1. **Resource Identification** → A URI uniquely identifies each resource. Example: /users/1001 identifies a specific user.
2. **Manipulation via Representations** → Clients modify resources by sending their representation (JSON/XML). Example: PUT /users/1001 with JSON body updates a user.
3. **Self-Descriptive Messages** → Requests and responses carry enough information (HTTP method, headers, content type) so clients/servers understand how to process them.
4. **HATEOAS (Hypermedia as the Engine of Application State)** → Responses guide clients with links to available next actions.

Example: A response for a book might include links like:

```
{
  "Id": 101,
  "Title": "REST Basics",
  "Links": [
    {"rel": "update", "href": "/books/101", "method": "PUT"},
    {"rel": "delete", "href": "/books/101", "method": "DELETE"}
  ]
}
```

Why it matters:

- Clients know exactly how to interact with the API.
- Reduces coupling between client and server.
- Easier for new clients to consume APIs.

Examples:

- GET <https://example.com/books> retrieves a list of books.
- GET <https://example.com/books/123> retrieves a book with ID 123.
- POST <https://example.com/books> creates a new book.
- PUT <https://example.com/books/123> updates the book with ID 123.
- DELETE <https://example.com/books/123> deletes the book with the specified ID.

Content Negotiation:

Clients and servers can negotiate the media type used for representing a resource. The client typically specifies its preferred format via the Accept header, and the server responds in one of the requested formats if supported.

Why It Matters:

- Allows different clients (mobile apps, browsers, etc.) to request the format they can best handle (e.g., JSON, XML, HTML).
- Servers can support multiple formats without changing the resource identifier.

Example: If a client requests an article by calling GET [/articles/456](#) and includes Accept: application/json, the server can respond with JSON (e.g., Content-Type: application/json). If the client had requested Accept: application/xml, the server could respond with XML if it supports it.

Layered System:

REST allows APIs to be built as a **layered architecture**. Each layer has its own responsibility (e.g., security, caching, load balancing), and clients are unaware of the number of layers that exist.

Why it matters:

- Scalability → Easy to add caching servers, gateways, or load balancers.
- Security → Middle layers can handle authentication or rate limiting.
- Flexibility → Layers can develop independently.

Example:

In an e-commerce app:

- **Layer 1:** Web Server handles client requests.
- **Layer 2:** Application Layer applies business logic.
- **Layer 3:** Database stores data.

Clients don't know about internal layering; they just see a single API endpoint.

Code on Demand (Optional)

A REST API may optionally send **executable code** to clients (e.g., JavaScript). This allows extending client functionality without additional requests. **This is an optional constraint used to add flexibility for the client.**

Why it matters:

- Reduces server load.
- Allows clients to perform operations locally.

Example: A server sends a JavaScript snippet with sorting logic for a product list. The client runs it locally, avoiding repeated server requests for sorting.

Difference Between ASP.NET Web API and ASP.NET Core Web API

ASP.NET Web API and ASP.NET Core Web API both enable developers to build **RESTful services**; **however**, they differ in terms of platform support, performance, features, and modern architecture. Let's look at each aspect in detail.

Difference Between ASP.NET Web API and ASP.NET Core Web API		
	ASP.NET Web API	ASP.NET Core Web API
Platform Support	• Built on .NET Framework. • Runs only on Windows	• Built on .NET Core, which is cross-platform
Performance	• More reliant on architecture	• More lightweight, high-performance
API Design Features	• No built-in advanced feature	• Built-in support for API Versioning • Response Caching • Response Compression
Configuration and Hosting	• Typically hosted in IIS • Uses web.config configuration	• IIS, Kestrel, Docker • Uses appsettings.jsconfiguration
Dependency Injection	• DI support, but not natively - Support not	• Built-in DI support
Middleware Support	• No native middleware support	• A modern middleware pipeline

Platform Support

ASP.NET Web API:

- Built on the **.NET Framework**.
- Runs only on **Windows** because the .NET Framework is tied to Windows OS.
- Example: If your organization uses only Windows servers with IIS, ASP.NET Web API can work fine.

ASP.NET Core Web API:

- Built on **.NET Core**, which is **cross-platform**.
- Can run on **Windows, Linux, and macOS**, giving more deployment flexibility.
- Example: If you want to host your APIs in Docker containers or deploy them on Linux cloud servers (like AWS, Azure, GCP), ASP.NET Core Web API is the right choice.

Note: ASP.NET Core Web API is preferred when you want **cross-platform** support and modern deployment options.

Performance

ASP.NET Web API:

- While efficient for its time, it relies on older architecture and lacks optimizations found in .NET Core.
- Less effective in handling very high request loads.

ASP.NET Core Web API:

- Built from scratch for **performance and scalability**.
- Uses **Kestrel web server** (highly optimized, asynchronous by default).
- Can handle **millions of requests per second** with lower resource usage.

Note: For **high-traffic applications** like e-commerce or streaming platforms, ASP.NET Core Web API is far superior.

API Design Features

ASP.NET Web API:

- Supports RESTful services but **lacks built-in advanced features**.
- Developers often rely on external libraries for things like:
 1. API versioning
 2. Response compression
 3. Caching

ASP.NET Core Web API:

- Provides **out-of-the-box support** for:
 1. API Versioning → handle multiple versions of your API (/api/v1/products, /api/v2/products).
 2. Response Caching → reduce server load by reusing responses.
 3. Response Compression → shrink data payload (gzip, Brotli).

Note: ASP.NET Core reduces dependency on third-party packages and makes APIs **more robust and easier to maintain**.

Configuration and Hosting

ASP.NET Web API:

- **Typically hosted in IIS** (Internet Information Services).
- Configuration handled mostly through **config** (XML-based, rigid).

ASP.NET Core Web API:

- **Flexible hosting options:**
 1. IIS
 2. Kestrel (default, lightweight server)
 3. Docker containers
 4. Self-hosted services
- Configuration can come from multiple sources:
 1. json (JSON-based, simpler)
 2. Environment variables
 3. Command-line arguments

Note: ASP.NET Core allows **modern cloud-native deployment** and **flexible configuration management**.

Dependency Injection

ASP.NET Web API:

- Supports DI but not **natively**.
- Developers often integrate external frameworks (Unity, Autofac, Ninject).

ASP.NET Core Web API:

- Has **built-in Dependency Injection (DI)**.
- Cleaner, more maintainable code with loosely coupled services.

Note: ASP.NET Core provides inbuilt **DI support**, making it easier to build testable, modular applications.

Middleware Support

ASP.NET Web API:

- No native middleware pipeline.
- Uses **HttpHandlers and HttpModules**, which are more complex to configure and extend.

ASP.NET Core Web API:

- Uses a **modern middleware pipeline**.
- Developers can plug in middleware components for:
 1. Logging
 2. Authentication
 3. Exception handling
 4. Request/Response modifications

Note: ASP.NET Core's middleware pipeline makes it **simpler, cleaner, and more flexible** to customize request handling.

Choosing Between ASP.NET Web API and ASP.NET Core Web API

Choose ASP.NET Web API if:

- You are maintaining a **legacy system** built on the .NET Framework.
- Your infrastructure is **Windows-only** and you don't plan cross-platform support.

Choose ASP.NET Core Web API if:

- You want **cross-platform** support (Windows, Linux, macOS).
- You need **modern performance, scalability, and cloud deployment**.
- You prefer **built-in features** like caching, versioning, and DI.
- You want to build **future-proof applications**.

In short: **ASP.NET Web API is old and Windows-only, while ASP.NET Core Web API is modern, cross-platform, lightweight, and built for today's high-performance, cloud-first applications.**

ASP.NET Core Web API is the ideal choice for building modern, scalable, and cross-platform web services. It offers superior performance, built-in support for dependency injection and middleware, and the flexibility to run on Windows, Linux, or macOS. As we move towards an API-first development world, mastering ASP.NET Core Web API is crucial for any developer seeking to build robust and future-ready backend services that function seamlessly across platforms and devices.

In the next article, I will discuss [HTTP \(HyperText Transport Protocol\)](#), specifically HTTP Verbs or Methods, HTTP Status Codes, HTTP Requests, and Responses. In this article, I give a brief introduction to the ASP.NET Core Web API Framework. I hope you enjoy this Introduction to ASP.NET Core Web API article.

Dot Net Tutorials

About the Author: Pranaya Rout

Pranaya Rout has published more than 3,000 articles in his 11-year career.

Pranaya Rout has very good experience with Microsoft Technologies, Including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.

7 thoughts on “Introduction to ASP.NET Core Web API”

AHMAD

MAY 12, 2024 AT 5:52 PM

Thank u very much , this article was very helpful to me

[Reply](#)

SAM

MAY 21, 2024 AT 4:34 PM

Thank you so much

[Reply](#)

AMIYA

MAY 21, 2024 AT 4:35 PM

Very nice tutorials

[Reply](#)

TUMIYA

JULY 18, 2024 AT 4:01 PM

This was very helpful

[Reply](#)

DANIEL

NOVEMBER 23, 2024 AT 8:45 PM

Thank you ! Amazing work.

[Reply](#)

DUC

FEBRUARY 19, 2025 AT 9:45 AM

ty

[Reply](#)

PHEELLO

MAY 1, 2025 AT 4:05 PM

The information is useful.

[Reply](#)

Privacy Preferences

We and our partners share information on your use of this website to help improve your experience. For more information, or to opt out click the Do Not Sell My Information button below.

[Consent](#)

[Do Not Sell My Information](#)